PRACTICAL – 5

Implementation of a knapsack problem using dynamic programming.

Aim:
To implement the 0/1 Knapsack Problem using Dynamic Programming and determine the maximum value that can be obtained by selecting items within a given knapsack capacity.

Objective:
The objective of this experiment is to understand and implement the 0/1 Knapsack Problem using the Dynamic Programming technique. It aims to determine the optimal selection of items based on their weights and values while ensuring that the total weight does not exceed the given capacity. The experiment also helps in understanding the concept of optimal substructure and overlapping subproblems used in dynamic programming.

Algorithm:
1. **Start.**
2. Readthe number of items n.
3. Readthe **weight** and **value** of each item.
4. Readthe maximum capacity W of the knapsack.
5. Createa DP table dp[n+1][W+1].
6. Initialize the first row and first column of the table to 0.
7. For each item i from 1 to n:
   - o For each capacity w from 1 to W:
   - o If the weight of the current item is less than or equal to w, calculate:
   - o dp[i][w] = max(value[i-1] + dp[i-1][w-weight[i-1]], dp[i-1][w])
   - o Otherwise: dp[i][w] = dp[i-1][w]
8. The value stored in dp[n][W] represents the **maximum value** that can be obtained.
9. Display the maximum value.
10. **Stop.**

Code:

```
def knapsack(weights, profits, capacity, n):
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(1, capacity + 1):

            if weights[i - 1] <= w:
                dp[i][w] = max(
                    profits[i - 1] + dp[i - 1][w - weights[i - 1]],
```

```
                dp[i - 1][w]
            )
        else:
            dp[i][w] = dp[i - 1][w]

    return dp[n][capacity]


n = int(input("Enter the number of items: "))

weights = []
profits = []

print("Enter weights:")
for i in range(n):
    weights.append(int(input()))

print("Enter profits:")
for i in range(n):
    profits.append(int(input()))

capacity = int(input("Enter knapsack capacity: "))

max_profit = knapsack(weights, profits, capacity, n)

print("\nMaximum Profit:", max_profit)

print("\nEnrollment Number: 92460118708")
```

Output:

```
PS C:\Users\penta\Downloads\DAA>  & 'C:\Users\penta\AppData\Loc
py-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '65060' '-
Enter the number of items: 5
Enter weights:
5
3
6
4
2
Enter profits:
2
4
6
3
2
Enter knapsack capacity: 4

Maximum Profit: 4

Enrollment Number: 92460118708
```

Time Complexity:

Case	*Time Complexity*	*Description*
Best Case	*O(nW)*	*DP table is filled for all items and capacities.*
Average Case	*O(nW)*	*Each item is checked for every capacity.*
Worst case	*O(nW)*	*All n × W table entries are calculated*

Where "**W**" is the knapsack capacity.

Space Complexity:

Space Complexity	*Description*
O(nW)	*Uses a 2D DP table to store solutions for items and capacities.*